

```
import { useState, useEffect } from "react";

const ANTHROPIC_API = "https://api.anthropic.com/v1/messages";

const styles = `
  @import url('https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: 'DM Mono', monospace;
  background: #0a0a0f;
  color: #e8e8f0;
  min-height: 100vh;
}

.app {
  min-height: 100vh;
  background: #0a0a0f;
  position: relative;
  overflow: hidden;
}

.bg-glow {
  position: fixed;
  top: -200px;
  left: 50%;
  transform: translateX(-50%);
  width: 600px;
  height: 600px;
  background: radial-gradient(circle, rgba(99,255,180,0.08) 0%, transparent 70%);
  pointer-events: none;
  z-index: 0;
}

.header {
  position: relative;
  z-index: 10;
  padding: 28px 24px 0;
  text-align: center;
}

.logo {
  font-family: 'Syne', sans-serif;
```

```
    font-weight: 800;
    font-size: 28px;
    letter-spacing: -1px;
    color: #63ffb4;
}

.logo span { color: #e8e8f0; }

.tagline {
    font-size: 11px;
    letter-spacing: 3px;
    text-transform: uppercase;
    color: #555570;
    margin-top: 4px;
}

.badge {
    display: inline-block;
    margin-top: 10px;
    background: rgba(99,255,180,0.1);
    border: 1px solid rgba(99,255,180,0.25);
    color: #63ffb4;
    font-size: 10px;
    letter-spacing: 2px;
    text-transform: uppercase;
    padding: 4px 12px;
    border-radius: 20px;
}

.main {
    position: relative;
    z-index: 10;
    padding: 24px 16px;
    max-width: 480px;
    margin: 0 auto;
}

.card {
    background: rgba(255,255,255,0.03);
    border: 1px solid rgba(255,255,255,0.08);
    border-radius: 16px;
    padding: 20px;
    margin-bottom: 16px;
}

.section-label {
    font-size: 10px;
```

```
    letter-spacing: 3px;
    text-transform: uppercase;
    color: #555570;
    margin-bottom: 12px;
}
```

```
textarea {
    width: 100%;
    background: rgba(0,0,0,0.4);
    border: 1px solid rgba(255,255,255,0.1);
    border-radius: 10px;
    color: #e8e8f0;
    font-family: 'DM Mono', monospace;
    font-size: 13px;
    line-height: 1.6;
    padding: 14px;
    resize: none;
    outline: none;
    transition: border-color 0.2s;
    height: 160px;
}
```

```
textarea:focus { border-color: rgba(99,255,180,0.4); }
textarea::placeholder { color: #333348; }
```

```
.mode-row {
    display: flex;
    gap: 8px;
    margin-bottom: 16px;
}
```

```
.mode-btn {
    flex: 1;
    padding: 10px 8px;
    border-radius: 10px;
    border: 1px solid rgba(255,255,255,0.08);
    background: transparent;
    color: #555570;
    font-family: 'DM Mono', monospace;
    font-size: 11px;
    letter-spacing: 1px;
    text-transform: uppercase;
    cursor: pointer;
    transition: all 0.2s;
}
```

```
.mode-btn.active {
```

```
background: rgba(99,255,180,0.12);
border-color: rgba(99,255,180,0.4);
color: #63ffb4;
}

.generate-btn {
  width: 100%;
  padding: 16px;
  background: #63ffb4;
  color: #0a0a0f;
  border: none;
  border-radius: 12px;
  font-family: 'Syne', sans-serif;
  font-weight: 700;
  font-size: 14px;
  letter-spacing: 1px;
  text-transform: uppercase;
  cursor: pointer;
  transition: all 0.2s;
  margin-top: 4px;
}

.generate-btn:hover { background: #7fffc4; transform: translateY(-1px); }
.generate-btn:disabled { opacity: 0.4; cursor: not-allowed; transform: none; }

.loading {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  padding: 32px;
  color: #555570;
  font-size: 12px;
  letter-spacing: 2px;
  text-transform: uppercase;
}

.spinner {
  width: 16px;
  height: 16px;
  border: 2px solid #333348;
  border-top-color: #63ffb4;
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
}

@keyframes spin { to { transform: rotate(360deg); } }
```

```

/* FLASHCARDS */

.flashcard-area { margin-bottom: 16px; }

.fc-counter {
  font-size: 10px;
  letter-spacing: 3px;
  text-transform: uppercase;
  color: #555570;
  text-align: center;
  margin-bottom: 12px;
}

.flashcard {
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.08);
  border-radius: 16px;
  padding: 32px 24px;
  text-align: center;
  cursor: pointer;
  min-height: 180px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  transition: all 0.2s;
  position: relative;
}

.flashcard:hover { border-color: rgba(99,255,180,0.2); }

.fc-side {
  font-size: 9px;
  letter-spacing: 3px;
  text-transform: uppercase;
  margin-bottom: 16px;
  padding: 3px 10px;
  border-radius: 20px;
}

.fc-side.front {
  background: rgba(99,255,180,0.1);
  color: #63ffb4;
  border: 1px solid rgba(99,255,180,0.2);
}

.fc-side.back {

```

```
background: rgba(180,120,255,0.1);
color: #c49fff;
border: 1px solid rgba(180,120,255,0.2);
}

.fc-text {
  font-family: 'Syne', sans-serif;
  font-size: 16px;
  font-weight: 600;
  line-height: 1.5;
  color: #e8e8f0;
}

.fc-hint {
  position: absolute;
  bottom: 12px;
  right: 16px;
  font-size: 9px;
  letter-spacing: 2px;
  text-transform: uppercase;
  color: #333348;
}

.fc-nav {
  display: flex;
  gap: 8px;
  margin-top: 12px;
}

.nav-btn {
  flex: 1;
  padding: 12px;
  background: rgba(255,255,255,0.04);
  border: 1px solid rgba(255,255,255,0.08);
  border-radius: 10px;
  color: #888898;
  font-family: 'DM Mono', monospace;
  font-size: 12px;
  cursor: pointer;
  transition: all 0.2s;
}

.nav-btn:hover:not(:disabled) {
  background: rgba(99,255,180,0.08);
  border-color: rgba(99,255,180,0.2);
  color: #63ffb4;
}
```

```
.nav-btn:disabled { opacity: 0.25; cursor: not-allowed; }
```

```
/* QUIZ */
```

```
.quiz-question {  
  font-family: 'Syne', sans-serif;  
  font-size: 15px;  
  font-weight: 600;  
  line-height: 1.5;  
  margin-bottom: 16px;  
  color: #e8e8f0;  
}
```

```
.quiz-q-num {  
  font-size: 10px;  
  letter-spacing: 3px;  
  text-transform: uppercase;  
  color: #63ffb4;  
  margin-bottom: 8px;  
}
```

```
.option-btn {  
  width: 100%;  
  text-align: left;  
  padding: 13px 16px;  
  background: rgba(0,0,0,0.3);  
  border: 1px solid rgba(255,255,255,0.08);  
  border-radius: 10px;  
  color: #aaaabc;  
  font-family: 'DM Mono', monospace;  
  font-size: 12px;  
  line-height: 1.5;  
  cursor: pointer;  
  transition: all 0.15s;  
  margin-bottom: 8px;  
  display: block;  
}
```

```
.option-btn:hover:not(:disabled) {  
  background: rgba(99,255,180,0.06);  
  border-color: rgba(99,255,180,0.25);  
  color: #e8e8f0;  
}
```

```
.option-btn.correct {  
  background: rgba(99,255,180,0.12);  
  border-color: rgba(99,255,180,0.5);  
}
```

```
    color: #63ffb4;
}

.option-btn.wrong {
    background: rgba(255,90,90,0.1);
    border-color: rgba(255,90,90,0.35);
    color: #ff8080;
}

.explanation {
    background: rgba(99,255,180,0.06);
    border: 1px solid rgba(99,255,180,0.15);
    border-radius: 10px;
    padding: 12px 14px;
    font-size: 12px;
    color: #8888a0;
    line-height: 1.6;
    margin-top: 4px;
    margin-bottom: 12px;
}

.score-card {
    text-align: center;
    padding: 32px 20px;
}

.score-big {
    font-family: 'Syne', sans-serif;
    font-size: 56px;
    font-weight: 800;
    color: #63ffb4;
    line-height: 1;
    margin-bottom: 8px;
}

.score-label {
    font-size: 11px;
    letter-spacing: 3px;
    text-transform: uppercase;
    color: #555570;
    margin-bottom: 24px;
}

.restart-btn {
    padding: 12px 28px;
    background: transparent;
    border: 1px solid rgba(99,255,180,0.3);
```



```

border-radius: 10px;
color: #63ffb4;
font-family: 'DM Mono', monospace;
font-size: 12px;
letter-spacing: 2px;
text-transform: uppercase;
cursor: pointer;
transition: all 0.2s;
}

.restart-btn:hover {
  background: rgba(99,255,180,0.1);
}

.error-box {
  background: rgba(255,90,90,0.08);
  border: 1px solid rgba(255,90,90,0.2);
  border-radius: 10px;
  padding: 14px 16px;
  color: #ff8080;
  font-size: 12px;
  line-height: 1.6;
  margin-bottom: 16px;
}

.progress-bar {
  height: 2px;
  background: rgba(255,255,255,0.06);
  border-radius: 2px;
  margin-bottom: 20px;
  overflow: hidden;
}

.progress-fill {
  height: 100%;
  background: #63ffb4;
  border-radius: 2px;
  transition: width 0.3s ease;
}
`;

async function callClaude(notes, mode) {
  const prompt = mode === "flashcards"
    ? `You are a study tool. From the notes below, create exactly 6 flashcards.
Return ONLY valid JSON with this structure, no markdown, no extra text:
{"flashcards":[{"front":"question or term","back":"answer or definition"}]}`

```

Notes:

`${notes}``

`: `You are a study tool. From the notes below, create exactly 5 multiple choice questions. Return ONLY valid JSON with this structure, no markdown, no extra text:`
`{"questions":[{"question":"...", "options":["A) ...", "B) ...", "C) ...", "D) ..."], "correct": 0}`
The "correct" field is the 0-based index of the correct option.

Notes:

`${notes}`;`

```
const response = await fetch(ANTHROPIC_API, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "claude-sonnet-4-20250514",
    max_tokens: 1000,
    messages: [{ role: "user", content: prompt }]
  })
});

if (!response.ok) throw new Error("API error " + response.status);
const data = await response.json();
const text = data.content.map(b => b.text || "").join("");
const clean = text.replace(/```json|```/g, "").trim();
return JSON.parse(clean);
}

// — Flashcard viewer —————
function FlashcardViewer({ cards, onReset }) {
  const [idx, setIdx] = useState(0);
  const [flipped, setFlipped] = useState(false);

  const go = (dir) => {
    setIdx(i => i + dir);
    setFlipped(false);
  };

  const card = cards[idx];
  return (
    <div className="flashcard-area">
      <div className="fc-counter">{idx + 1} / {cards.length}</div>
      <div className="progress-bar">
        <div className="progress-fill" style={{ width: `${((idx + 1) / cards.length) * 100}%`}>
        </div>
      </div>
      <div className="flashcard" onClick={() => setFlipped(f => !f)}>
        <span className={`fc-side ${flipped ? "back" : "front"}`>
          {flipped ? "Answer" : "Question"}
        </span>
      </div>
    </div>
  );
}
```

```

        </span>
        <div className="fc-text">{flipped ? card.back : card.front}</div>
        <span className="fc-hint">tap to flip</span>
      </div>
      <div className="fc-nav">
        <button className="nav-btn" onClick={() => go(-1)} disabled={idx === 0}>-
        <button className="nav-btn" onClick={onReset}>New</button>
        <button className="nav-btn" onClick={() => go(1)} disabled={idx === cards
      </div>
    </div>
  );
}

// — Quiz viewer —————
function QuizViewer({ questions, onReset }) {
  const [idx, setIdx] = useState(0);
  const [chosen, setChosen] = useState(null);
  const [score, setScore] = useState(0);
  const [done, setDone] = useState(false);

  const q = questions[idx];

  const pick = (i) => {
    if (chosen !== null) return;
    setChosen(i);
    if (i === q.correct) setScore(s => s + 1);
  };

  const next = () => {
    if (idx < questions.length - 1) {
      setIdx(i => i + 1);
      setChosen(null);
    } else {
      setDone(true);
    }
  };

  if (done) return (
    <div className="card score-card">
      <div className="score-big">{score}/{questions.length}</div>
      <div className="score-label">Score</div>
      <button className="restart-btn" onClick={onReset}>Try Again</button>
    </div>
  );

  return (
    <div className="card">

```

```

    <div className="progress-bar">
      <div className="progress-fill" style={{ width: `${((idx + 1) / questions.
    </div>
    <div className="quiz-q-num">Q{idx + 1} of {questions.length}</div>
    <div className="quiz-question">{q.question}</div>
    {q.options.map((opt, i) => (
      <button
        key={i}
        className={`option-btn ${chosen !== null ? (i === q.correct ? "correct"
        onClick={() => pick(i)}
        disabled={chosen !== null}
      >
        {opt}
      </button>
    )}}
    {chosen !== null && (
      <>
        <div className="explanation">{q.explanation}</div>
        <button className="nav-btn" style={{ width: "100%", marginTop: 0 }} onC
          {idx < questions.length - 1 ? "Next →" : "See Score"}
        </button>
      </>
    )}
  </div>
);
}

```

```

// — Main App —————
export default function StudySnap() {
  const [notes, setNotes] = useState("");
  const [mode, setMode] = useState("flashcards");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [result, setResult] = useState(null);

  const generate = async () => {
    if (!notes.trim() || notes.trim().length < 20) {
      setError("Please paste at least a few sentences of notes.");
      return;
    }
    setError(null);
    setLoading(true);
    setResult(null);
    try {
      const data = await callClaude(notes, mode);
      setResult({ mode, data });
    } catch (e) {

```

```

    setError("Something went wrong. Check your notes and try again.");
  } finally {
    setLoading(false);
  }
};

const reset = () => { setResult(null); setError(null); };

return (
  <>
    <style>{styles}</style>
    <div className="app">
      <div className="bg-glow" />
      <div className="header">
        <div className="logo">Study<span>Snap</span></div>
        <div className="tagline">AI-powered study tools</div>
        <div className="badge">♦ Free Beta</div>
      </div>

      <div className="main">
        {!result && (
          <div className="card">
            <div className="section-label">01 – Paste your notes</div>
            <textarea
              value={notes}
              onChange={e => setNotes(e.target.value)}
              placeholder="Paste your lecture notes, textbook excerpt, or any s
            />
            <div className="section-label" style={{ marginTop: 20 }}>02 – Choos
            <div className="mode-row">
              <button
                className={`mode-btn ${mode === "flashcards" ? "active" : ""}`}
                onClick={() => setMode("flashcards")}
              >0 Flashcards</button>
              <button
                className={`mode-btn ${mode === "quiz" ? "active" : ""}`}
                onClick={() => setMode("quiz")}
              >♦ Quiz Me</button>
            </div>
            {error && <div className="error-box">{error}</div>}
            <button className="generate-btn" onClick={generate} disabled={loadi
              {loading ? "Generating..." : "Generate →"}
            </button>
          </div>
        )}

        {loading && (

```

```
        <div className="loading">
            <div className="spinner" /> Analyzing notes...
        </div>
    )}

    {result?.mode === "flashcards" && result.data.flashcards && (
        <FlashcardViewer cards={result.data.flashcards} onReset={reset} />
    )}

    {result?.mode === "quiz" && result.data.questions && (
        <QuizViewer questions={result.data.questions} onReset={reset} />
    )}
</div>
</div>
</>
);
}
```